The Justified Intent & Refinement Ledger

Stratified Justification, Compositional Assurance, and Coherent Software Evolution

Section titled “Stratified Justification, Compositional Assurance, and Coherent Software Evolution”

Formal Technical Report · JIRL–TR–2026–01 · 8 November 2026

Abstract

Section titled “Abstract”

Software evolution changes premises, requirements, design choices, and the evidence used to justify them alongside program text. A dependency graph alone cannot determine whether an alternative justification survives, whether an earlier check establishes a strengthened obligation, or whether individually acceptable branch changes remain acceptable together. We introduce the Justified Intent & Refinement Ledger (JIRL), a finite mathematical model and persistent reference architecture separating material definitions, evaluation and authority, and published snapshots. Grouped positive justifications express conjunction within a reason and disjunction across reasons; inconsistency is evaluated on full derived configurations rather than incorporated into local inference guards. We prove least–fixed–point and order–independence results, characterize consistent assumption environments and their minimal labels, and establish exponential explicit–label lower bounds. Finite counterexamples refute topology–only verification boundaries, artifact–hash reuse, unverified interface–hash cutoff, and per–edge newest–version traversal. An obligation–first calculus connects joint exploration, interface compatibility, non–vacuous circular assume–guarantee reasoning, vertical simulation, and code realization. Incremental evaluation is proved equivalent to cold evaluation when complete observational projections and reuse certificates preserve the defined verdict projection. A four–phase publication protocol installs an evaluated and authorized candidate under an atomic expected–head update. Complexity analysis separates the inexpensive finite kernel from potentially dominant proof and authoring costs, while finite witness checks and a four–arm falsification protocol separate mathematical soundness from empirical evaluation of engineering utility.

1. Introduction and problem formulation

Section titled “1. Introduction and problem formulation”

Consider a service designed under the assumption that no tenant–specific rule exists. Two premises jointly justify a shared cache; a different mechanism could justify the same cache independently. A new tenant rule defeats the absence assumption. At the same time, a performance requirement is strengthened and two branches adjust different parts of a capacity budget. Tracking only modified source files leaves four engineering questions unanswered: which justification still applies, which check is now inadequate, which behavioral interaction needs joint verification, and which coherent candidate may replace the accepted design?

JIRL addresses these as four distinct boundaries:

1. **Justification under change.** A conclusion may require several premises jointly and may admit several alternative derivations. Retracting one premise removes one derivation route while leaving independent routes intact. Absence assumptions vary nonmonotonically across inventory changes, whereas positive inference within a fixed candidate remains monotone.

2. **Intent versus behavioral verification.** Shared rationale and shared runtime state are orthogonal relations: components linked by a common goal can execute independently, and components without direct rationale links can contend for shared resources. Requirements, contracts, state machines, and programs require explicit satisfaction and realization relations beyond graph connectivity.

3. **Persistence and coherent observation.** Following the newest revision of each dependency independently can synthesize a configuration that was never evaluated together, while embedding current evidence inside its own evaluated content creates a cyclic hash dependency. Historical object identity, current evaluation evidence, and named accepted snapshots are therefore stratified.

4. **Evolution versus synthesis.** An intent change ΔI rarely determines a unique implementation. The transition

$$G_{t+1} = \text{evolve}(G_t, \Delta I, \pi, \alpha; \omega)$$
Gt+1 = evolve(Gt, ΔI, π, α; ω)

is defined relative to an explicit interpretation and selection policy ππ, authority policy αα, and external checker/environment observations ωω, yielding either a published snapshot or an explicit phase diagnostic (§5.2).

1.1 Scope, assumptions, and contribution

Section titled “1.1 Scope, assumptions, and contribution”

This report specifies a mathematical model and reference architecture. The finite kernel assumes immutable content-addressed inputs, declared dependency and model horizons, trusted assurance and authority providers, collision-free content identifiers over the objects used, and a publication store supporting atomic expected-head updates with durable transition records. Proofs, finite tests, and governance judgments carry distinct assurance modes: each establishes the proposition, exercised scope, or authorization recorded in its own obligation. External deployments and data migrations are governed as separate actions outside snapshot publication.

The core contribution is the composition of four boundaries—positive support, scoped assurance, action-specific authority, and coherent persistence—connected through explicit obligations. Transformation certificates from design-maintenance systems [13], where a change is commuted through a verified derivation with an applicability argument, fit naturally as one class of obligation evidence.

1.2 Theoretical placement

Section titled “1.2 Theoretical placement”

Truth-maintenance systems motivate explicit assumption environments and nogoods, while positive provenance distinguishes joint premises from alternative derivations [1,2]. JIRL uses finite Boolean support: the meaning of a derivation step is carried by a versioned rule specification and its bound verification obligations.

Goal-oriented requirements engineering separates desired properties from system specifications and domain assumptions [3]. Axiomatic Design’s zigzagging between functional requirements and design parameters motivates **mechanism-conditioned** subgoals [4]: selecting a replicated storage mechanism introduces consistency and recovery obligations beyond the abstract availability goal. Design Structure Matrix (DSM) analysis exposes structural coupling to guide partitioning, while semantic compatibility is discharged by behavioral composition obligations [5].

IBIS and QOC preserve design questions, options, qualitative assessments, and selection rationale [6]. Interface automata and assume-guarantee calculi supply compositional behavioral rules [7,8], and refinement mappings connect abstract and concrete transition systems [9]. Persistent data structures and self-adjusting computation provide versioned structural sharing and from-scratch incremental consistency criteria [10,11]. JIRL connects these traditions at explicit obligations so that rationale maintenance, compositional verification, and snapshot publication share a single dependency and applicability model; Section 6 specifies an empirical protocol to test when that integration yields net engineering benefit.

2. Abstract model and stratified persistence

Section titled “2. Abstract model and stratified persistence”

2.1 Material, evaluation, authority, and snapshots

Section titled “2.1 Material, evaluation, authority, and snapshots”

A finite material is an eight-tuple

$$M = (C, R, J, N, O, D, R_C, R_O),$$
M = (C, R, J, N, O, D, RC, RO),

where CC is the selected set of exact claim revisions, $R \subseteq C$R ⊆ C the admitted roots, JJ the grouped justifications, NN the applicable nogoods, OO the obligations, DD the decision records, and $R_C \subseteq C$RC ⊆ C, $R_O \subseteq O$RO ⊆ O the required achievements and checks. All

references resolve to active members of MM, with at most one active revision per claim or obligation lineage. Lineages provide stable navigation across history, while snapshots select exact immutable revisions.

A claim states a commitment, assumption, absence condition, proposition, artifact description, or selection. Each justification $j \in J$j ∈ J and nogood $n \in N$n ∈ N has the form

$$j = (P_j, h_j, \rho_j) \quad \text{with } P_j \subseteq C,\ h_j \in C, \qquad K_n \subseteq C,$$
$$\text{j} = (\text{Pj}, \text{hj}, \text{ρj}) \quad \text{with Pj} \subseteq \text{C},\ \text{hj} \in \text{C}, \qquad \text{Kn} \subseteq \text{C},$$

where P_jPj is a finite conjunctive premise set, $h_j$hj is the consequent, and ρ_jρj is an immutable rule specification that defines any role-sensitive premise ordering. Multiple justifications with the same consequent $h_j$hj represent alternative derivations. A nogood nn declares a subset $K_n \subseteq C$Kn ⊆ C that cannot co-hold in a consistent configuration; $K_n = \varnothing$Kn = ∅ denotes unconditional inconsistency.

An obligation pins a subject, requested property and evidence scope, evaluator identity, and configuration. A check request binds an obligation to an exact MM and a consumed-environment manifest. An evaluation EE selects applicable check outcomes under an assurance profile. An authority decision AA answers an exact publication request over base snapshots, MM, EE, principal, policy, intent, interpretation, and protected-item dispositions. A snapshot SS binds exact references to its parent snapshots, MM, EE, and AA.

```
Material stratum          Assessment / authority stratum          Snapshot stratum
payloads, claims          CheckRequests -> Evidence               old parents
edges, constraints  --->  Evaluation -> AuthorityRequest   --->    + Material
decisions, obligations    -> AuthorityDecision                     + Evaluation
-> Material                                                        + AuthorityDecision
                                                                      |
                                                  named ref --atomic expected-head update
```

Arrows in the diagram show construction prerequisites; stored pointers in the assessment and snapshot strata target previously constructed objects in earlier strata. Within the middle stratum, evaluation precedes publication authorization: earlier judgment evidence may enter as a pinned evaluation input, whereas the publication authority decision AA refers to the completed evaluation EE.

2.2 Bottom-up content identity and constant-operation root resolution

Section titled “2.2 Bottom-up content identity and constant-operation root resolution”

Objects use a canonical byte encoding and domain-separated cryptographic hash, with content identifiers derived from the hashed payload. Keys are canonically sorted; arrays representing mathematical sets are sorted and duplicate-free; sequences retain semantic order; and blobs preserve exact bytes. Boundary validation checks schema conformance, content identity, reference closure, and active membership on load.

Lemma 1 (Acyclic construction). If every typed object reference points to an earlier object in a well-founded construction order, content identifiers can be computed bottom-up without self-referential hash equations.

Proof. A leaf object’s encoding contains no unresolved references, so its hash determines its identifier directly. By induction, when all earlier objects in the construction order have assigned identifiers, substituting those identifiers into the next object’s reference fields yields a complete finite byte encoding and hence a well-defined hash. Claims are constructed prior to separately stored justification records, so cycles in logical support do not induce cycles in typed object references; snapshots reference only existing parent snapshots. Any topological order of the finite typed reference graph supplies the required construction rank. □□

Lemma 2 (Pinned snapshot resolution). Under a store cost model counting indexed lookups, resolving a named ref to its root snapshot envelope requires $O(1)$O(1) store operations (one atomic ref read and one envelope fetch, with byte transfer proportional to envelope size), and subsequent traversals along exact immutable references remain isolated from concurrent ref updates; enumerating a referenced closure of kk objects takes $O(k)$O(k) store operations.

Proof. The atomic ref read returns a single snapshot identifier ss, and fetching ss retrieves its immutable root envelope. Because every internal reference targets an exact content identifier and retained objects are immutable, traversal from ss accesses the same object

bytes regardless of concurrent updates to the mutable named ref. □□

2.3 Decisions and the epistemic/governance boundary

Section titled “2.3 Decisions and the epistemic/governance boundary”

For a design question qq, a decision record pairs each candidate option $\mathrm{opt}(q, o_i)$opt(q, oi) with a distinct selection proposition $\mathrm{sel}(q, o_i)$ sel(q, oi) stating that option $o_i$oi is chosen. Option feasibility is established by separate evidence claims bound to $\mathrm{opt}(q, o_i)$opt(q, oi), allowing multiple feasible options (including rejected alternatives) to remain recorded simultaneously. Qualitative assessment marks (`++`, `+`, `0`, `-`, `--`) record structured trade-offs unless a quantitative utility obligation is separately specified.

A configuration WW satisfies the at-most-one selection invariant when

$$\left|\{i : \mathrm{sel}(q, o_i) \in W\}\right| \leq 1.$$
$$\left|\{\mathsf{i} : \mathsf{sel}(\mathsf{q}, \mathsf{oi}) \in \mathsf{W}\}\right| \leqslant 1.$$

When a decision record designates a selected option, its corresponding $\mathrm{sel}(q, o_i)$sel(q, oi) must belong to WW; when the active assurance profile requires the question to be resolved, exactly one selection proposition must hold:

$$\left|\{i : \mathrm{sel}(q, o_i) \in W\}\right| = 1.$$
$$\left|\{\mathsf{i} : \mathsf{sel}(\mathsf{q}, \mathsf{oi}) \in \mathsf{W}\}\right| = 1.$$

Unresolved questions (00 selected options) are valid in exploratory snapshots. While the at-most-one constraint can be expressed via pairwise nogoods $\{\mathrm{sel}(q, o_i), \mathrm{sel}(q, o_k)\}${sel(q, oi), sel(q, ok)} ($i \neq k$i = k), the at-least-one resolution requirement is a positive obligation: because the family of nogood-consistent sets is downward-closed (Theorem 2(i)), no set of nogoods over positive claims can exclude the empty selection while admitting a singleton selection.

Factual assessments establish empirical or analytical properties of an option, whereas option selection records a policy judgment and snapshot publication records action-specific governance authority. Authority is modeled as a coverage relation between principals and request scopes rather than a total scalar rank: two principals authorized for disjoint scopes AA and BB are incomparable, and replacing or retiring a protected item requires authority covering that specific disposition. Each rejected option records a human-readable `reopenWhen` condition for review; automated reopening, when desired, is attached as a separate query obligation with a formal predicate.

2.4 Stratified scoped absence queries

Section titled “2.4 Stratified scoped absence queries”

Let $I_\sigma(M)$Iσ(M) denote the finite static material inventory under a declared inclusion scope σσ, and let qq be a pinned predicate over claim metadata or payloads. Its match set is

$$Q_{q,\sigma}(M) = \{c \in I_\sigma(M) : q(c)\}.$$
$$\mathsf{Q}_{\mathsf{q},\sigma}(\mathsf{M}) = \{\mathsf{c} \in \mathsf{I}\sigma(\mathsf{M}) : \mathsf{q}(\mathsf{c})\}.$$

A query report records the evaluated material, predicate, traversed scope, match set, and completion status. Discharging an absence claim requires a `complete` scan over σσ with $Q_{q,\sigma}(M) = \varnothing$Qq,σ(M) = ∅; a stopped scan yields an `incomplete` outcome, while any single matching element $c \in Q_{q,\sigma}(M)$c ∈ Qq,σ(M) suffices to refute absence immediately. Because an empty match set depends on the collection membership of $I_\sigma(M)$Iσ(M), subsequent insertion into σσ requires re-evaluation unless a narrower observational projection certifies non-interference.

Evaluation is stratified in the order: static inventory $I_\sigma(M)$ → Iσ(M) → query and check outcomes → → frozen rule eligibility → → warranted positive closure $W(M, E)$W(M, E). Queries inspect only the static material stratum and never read their own current warrant or

the closure WW being computed. Without stratification, allowing an absence claim aa to hold iff no warranted claim satisfies PP, while a claim cc satisfying PP is warranted iff aa holds, yields

$$w(a) \iff \neg w(a),$$
$$\mathrm{w(a)} \iff \neg \mathrm{w(a)},$$

which admits no two-valued fixed point. Restricting queries to the static inventory eliminates this cycle while preserving fixed-candidate monotonicity: within a fixed (M, E)(M, E), the positive closure operator is monotone (Theorem 1), whereas across distinct materials $M \to M'$M → M′, inserting a matching claim invalidates the prior empty-query evidence and retracts downstream warrant.

2.5 Reference data model

Section titled “2.5 Reference data model”

The TypeScript definitions below specify the core reference records quantified over in §§3–5. Content-addressed references `Ref<K>` carry a compile-time phantom brand `K` to prevent cross-kind substitution, while runtime boundary validation enforces schema conformance and reference closure. Process-level execution diagnostics (exit codes, signals, wall-clock timeouts) belong to checker adapter metadata outside the core semantic report.

```ts
declare const refBrand: unique symbol;
export type RefKind =
  | "blob" | "claim" | "justification" | "nogood" | "obligation"
  | "decision" | "material" | "check-request" | "check-report" | "evidence"
  | "evaluation" | "authority-request" | "authority-decision" | "snapshot";
export type Ref<K extends RefKind> = string & {readonly [refBrand]: K};
export type Lineage = string;
export type Scope = string;

export interface Claim {
  readonly schema: "intent-ledger/claim/2";
  readonly lineage: Lineage;
  readonly kind: "commitment" | "assumption" | "absence" | "proposition" | "artifact" | "selection";
  readonly scope: Scope;
  readonly statement: string;
  readonly payload?: {readonly format: string; readonly content: Ref<"blob">};
  readonly tags: Readonly<Record<string, string>>;
}

export interface Justification {
  readonly schema: "intent-ledger/justification/2";
  readonly premises: readonly Ref<"claim">[];
  readonly consequent: Ref<"claim">;
  readonly rule: Ref<"blob">; // Versioned inference or refinement rule specification.
}

export interface Nogood {
  readonly schema: "intent-ledger/nogood/2";
  readonly members: readonly Ref<"claim">[]; // Jointly incompatible claim set within one configuration.
  readonly reason: string;
}

export type CheckSubject =
  | {readonly kind: "claim"; readonly claim: Ref<"claim">}
  | {readonly kind: "justification"; readonly justification: Ref<"justification">}
  | {readonly kind: "candidate"};

export interface Obligation {
  readonly schema: "intent-ledger/obligation/2";
  readonly lineage: Lineage;
  readonly scope: Scope;
  readonly subject: CheckSubject;
  readonly mode: "judgment" | "proof" | "test" | "query";
  readonly specification: Ref<"blob">; // Requested property, assumptions, and evidence scope.
  readonly checker: Ref<"blob">;       // Pinned evaluator/adapter identity and protocol.
  readonly configuration: Ref<"blob">; // Pinned parameters, bounds, and solver settings.
}

export interface DecisionRecord {
  readonly schema: "intent-ledger/decision/2";
  readonly question: Ref<"claim">;
  readonly options: readonly {
    readonly option: Ref<"claim">;
    readonly selection: Ref<"claim">; // Proposition stating that this option is selected.
  }[];
  readonly assessments: readonly {
    readonly criterion: Ref<"claim">;
    readonly option: Ref<"claim">;
    readonly assessment: Ref<"claim">;
    readonly mark?: "++" | "+" | "0" | "-" | "--";
  }[];
  readonly selectedOption: Ref<"claim"> | null; // Selected option, or null when the question remains open.
  readonly rejected: readonly {
    readonly option: Ref<"claim">;
    readonly reason: string;
    readonly reopenWhen: string; // Human-readable review condition for reconsidering the option.
  }[];
}

export interface Material {
```

```
  readonly schema: "intent-ledger/material/2";
  readonly claims: readonly Ref<"claim">[];
  readonly roots: readonly Ref<"claim">[];
  readonly justifications: readonly Ref<"justification">[];
  readonly nogoods: readonly Ref<"nogood">[];
  readonly decisions: readonly Ref<"decision">[];
  readonly obligations: readonly Ref<"obligation">[];
  readonly requiredObligations: readonly Ref<"obligation">[];
  readonly requiredClaims: readonly Ref<"claim">[];
}

export interface CheckRequest {
  readonly schema: "intent-ledger/check-request/2";
  readonly material: Ref<"material">;
  readonly obligation: Ref<"obligation">;
  readonly environment: Ref<"blob">; // Complete manifest of consumed external inputs.
}

export interface QuerySpecification {
  readonly schema: "intent-ledger/query-specification/1";
  readonly view: "claim-inventory";
  readonly scope: Scope;
  readonly predicate: Ref<"blob">;
  readonly expectation:
    | {readonly kind: "empty"}
    | {readonly kind: "exact"; readonly matches: readonly Ref<"claim">[]}
    | {readonly kind: "predicate"; readonly predicate: Ref<"blob">};
}

export type CheckResult =
  | {readonly kind: "passed"} // Discharges the pinned request's property within its declared scope.
  | {readonly kind: "refuted"; readonly counterexample: Ref<"blob">}
  | {readonly kind: "incomplete"; readonly reason: string; readonly partial: Ref<"blob"> | null}
  | {readonly kind: "unsupported"; readonly reason: string}
  | {readonly kind: "error"; readonly diagnostic: Ref<"blob">}
  | {readonly kind: "not_run"; readonly reason: string};

export type CheckReport = {
  readonly schema: "intent-ledger/check-report/1";
  readonly request: Ref<"check-request">;
  readonly result: CheckResult;
  readonly exercised: readonly CheckSubject[];
  readonly omitted: readonly CheckSubject[];
  readonly completeness: "complete" | "incomplete";
  readonly details: Ref<"blob">; // Adapter-specific proof certificate, trace, or execution log.
} & (
  | {readonly kind: "check"}
  | {readonly kind: "query"; readonly matches: readonly Ref<"claim">[]}
);

export interface EvidenceRecord {
  readonly schema: "intent-ledger/evidence/2";
  readonly report: Ref<"check-report">;
  readonly basis: readonly Ref<"evidence">[]; // Prior evidence records cited by a reuse certificate.
}

export interface Evaluation {
  readonly schema: "intent-ledger/evaluation/2";
  readonly material: Ref<"material">;
  readonly profile: Ref<"blob">; // Pinned assurance and checker-eligibility policy.
  readonly environment: Ref<"blob">;
  readonly evidence: readonly Ref<"evidence">[];
}

export type AcceptedItem =
  | {readonly collection: "claims" | "roots" | "requiredClaims"; readonly ref: Ref<"claim">}
  | {readonly collection: "obligations" | "requiredObligations"; readonly ref: Ref<"obligation">}
  | {readonly collection: "justifications"; readonly ref: Ref<"justification">}
  | {readonly collection: "decisions"; readonly ref: Ref<"decision">}
  | {readonly collection: "nogoods"; readonly ref: Ref<"nogood">};
```

```
export interface AcceptedDisposition {
  readonly before: AcceptedItem;
  readonly after: AcceptedItem | null; // Replacement item, or null when withdrawing the protected role.
  readonly reason: string;
}

export interface AuthorityRequest {
  readonly schema: "intent-ledger/authority-request/2";
  readonly bases: readonly Ref<"snapshot">[];
  readonly material: Ref<"material">;
  readonly evaluation: Ref<"evaluation">;
  readonly policy: Ref<"blob">;
  readonly principal: string;
  readonly intent: Ref<"blob">;
  readonly interpretation: Ref<"blob">;
  readonly dispositions: readonly AcceptedDisposition[];
  readonly action: "publish_snapshot";
}

export interface AuthorityDecision {
  readonly schema: "intent-ledger/authority-decision/2";
  readonly request: Ref<"authority-request">;
  readonly provider: Ref<"blob">;
  readonly recordedAt: string; // Provider-recorded UTC timestamp.
  readonly result: "authorized" | "denied" | "indeterminate";
  readonly receipt: Ref<"blob">;
}

export interface Snapshot {
  readonly schema: "intent-ledger/snapshot/2";
  readonly parents: readonly Ref<"snapshot">[];
  readonly material: Ref<"material">;
  readonly evaluation: Ref<"evaluation">;
  readonly authorization: Ref<"authority-decision">;
}
```

Reference arrays represent canonical sets except for `parents` and `bases`, which are ordered duplicate–free sequences whose first element identifies the expected publication head. Within a `Material`, claims and obligations have unique active lineages; every root, justification endpoint, nogood member, required claim, and non–candidate obligation subject belongs to the active material, and `requiredObligations` is a subset of `obligations`. Each question has at most one active `DecisionRecord`, with disjoint selected and rejected option sets. An `Evaluation` selects at most one outcome per exact request; conflicting outcomes for the same deterministic request require explicit reconciliation, and prior `basis` evidence applies to a new material only when accompanied by a current reuse report.

3. Support algebra and consistency semantics

Section titled “3. Support algebra and consistency semantics”

3.1 Positive closure and global consistency

Section titled “3.1 Positive closure and global consistency”

Define the one–step positive consequence operator

$$\mathrm{Pos}(U) = \{h_j \mid j \in J \text{ and } P_j \subseteq U\}.$$

Pos(U) = {hj ∣ j ∈ J and Pj ⊆ U}.

Consider first the locally guarded operator that filters each rule application against nogoods NN:

$$T_R(U) = R \cup U \cup \left\{h_j \;\middle|\; j \in J,\ P_j \subseteq U \text{ and no } K_n \subseteq U \cup \{h_j\}\right\}.$$

TR(U) = R ∪ U ∪ {hj ∣ j ∈ J, Pj ⊆ U and no Kn ⊆ U ∪ {hj}}.

Proposition 1. There exist finite materials for which T_RTR is nonmonotone, parallel step evaluation activates a nogood, and no least fixed point above RR exists.

Proof. Let

$$C = \{a, b, c\}, \quad R = \{a\},$$
$$J = \big\{(\{a\}, b),\ (\{a\}, c)\big\}, \quad N = \big\{\{b, c\}\big\}.$$
$$\mathsf{CJ = \{a, b, c\},\ = \{(\{a\}, b),\ (\{a\}, c)\}, RN = \{a\},\ = \{\{b, c\}\}.}$$

For $U = \{a\} \subset V = \{a, c\}$U = {a} ⊂ V = {a, c}, both individual conclusions pass the local guard at UU, whereas bb is blocked at VV:

$$T_R(U) = \{a, b, c\} \not\subseteq \{a, c\} = T_R(V).$$
$$\mathsf{TR(U) = \{a, b, c\} \qquad \{a, c\} = TR(V).}$$

Evaluating $T_R(U)$TR(U) in parallel derives the forbidden pair $\{b, c\}${b, c}. The fixed points of T_RTR containing RR are $\{a, b\}${a, b}, $\{a, c\}${a, c}, and $\{a, b, c\}${a, b, c}; their intersection is $\{a\}${a}, which is not a fixed point of T_RTR, so no least fixed point above RR exists. Serializing rule applications makes the resulting fixed point depend on evaluation order. □□

3.2 Grounded and warranted closures

Section titled “3.2 Grounded and warranted closures”

JIRL therefore separates positive derivation from conflict detection. Define the unguarded positive operator and grounded closure by

$$F_M(U) = R \cup U \cup \mathrm{Pos}(U), \qquad D(M) = \mathrm{lfp}(F_M).$$
$$\mathsf{FM(U) = R \cup U \cup Pos(U), \qquad D(M) = lfp(FM).}$$

Before computing warrant, freeze the evaluation EE. Let $B(x)$B(x) denote the active obligations bound to subject xx, and let $\mathrm{Pass}(x)$ Pass(x) hold when every obligation in $B(x)$B(x) has an applicable `passed` report admitted by the assurance profile (for an admitted root rr with $B(r) = \varnothing$B(r) = ∅, $\mathrm{Pass}(r)$Pass(r) holds vacantly as a declared commitment; for a justification jj, eligibility requires $B(j) \neq \varnothing$ B(j) ≠ ∅, $\mathrm{Pass}(j)$Pass(j), and $\mathrm{Pass}(h_j)$Pass(hj)). Define the eligible roots and eligible justifications by

$$R_E = \{r \in R \mid \mathrm{Pass}(r)\},$$
$$J_E = \{j \in J \mid B(j) \neq \varnothing \wedge \mathrm{Pass}(j) \wedge \mathrm{Pass}(h_j)\},$$
$$\mathsf{RE\,JE = \{r \in R \mid Pass(r)\},\ = \{j \in J \mid B(j) \neq \varnothing \wedge Pass(j) \wedge Pass(hj)\},}$$

and the warranted operator and warranted closure by

$$F_{M,E}(U) = R_E \cup U \cup \{h_j \mid j \in J_E,\ P_j \subseteq U\},$$
$$W(M, E) = \mathrm{lfp}(F_{M,E}).$$
$$\mathsf{FM,E\,(U)\,W\,(M, E) = RE \cup U \cup \{hj \mid j \in JE,\ Pj \subseteq U\},\ = lfp(FM,E).}$$

Because R_ERE and J_EJE depend only on the frozen evaluation EE, eligibility is independent of UU.

Theorem 1. Both F_MFM and $F_{M,E}$FM,E are monotone and inflationary on $(2^C, \subseteq)$(2C, ⊆), admit a unique least fixed point, and reach it via any fair positive-rule worklist in order-independent $O(|C| + |J| + P)$O(|C| + |J| + P) time and space (where $P = \sum_{j \in J} |P_j|$|P| = Σj∈J |Pj|), excluding check execution and input decoding.

Proof. For $U \subseteq V$U ⊆ V, $P_j \subseteq U \implies P_j \subseteq V$Pj ⊆ U ⟹ Pj ⊆ V. Since each operator's root set and eligible rule set are fixed, every element generated at UU is also generated at VV, establishing monotonicity; union with UU establishes inflationarity. On the finite powerset lattice $(2^C, \subseteq)$(2C, ⊆), the ascending Kleene chain

$$X_0 = \varnothing, \qquad X_{k+1} = F(X_k)$$
$$\mathsf{X0} = \varnothing, \qquad \mathsf{Xk{+}1} = \mathsf{F(Xk)}$$

stabilizes in at most $|C|$|C| strict steps at a fixed point X_*X∗. For any fixed point YY, induction gives $X_0 \subseteq Y$X0 ⊆ Y and $X_{k+1} = F(X_k) \subseteq F(Y) = Y$Xk+1 = F(Xk) ⊆ F(Y) = Y, so X_*X∗ is the unique least fixed point.

To compute X_*X∗ in linear time, initialize for each rule jj a counter $c_j = |P_j|$cj = |Pj| of unmet premises and for each claim an adjacency list of incident rules. Enqueue all roots and fire any zero-premise rules ($P_j = \varnothing$Pj = ∅), enqueueing each newly reached claim at most once. Dequeueing a claim pp decrements $c_j$cj for each rule jj with $p \in P_j$p ∈ Pj; when $c_j$cj reaches 00, rule jj fires and enqueues $h_j$hj if unseen. By induction, every enqueued claim belongs to X_*X∗, and upon queue drainage every rule whose premises lie in the reached set has fired. The reached set is therefore a fixed point containing the roots and contained in X_*X∗, hence equal to X_*X∗ regardless of queue discipline. Initialization, counter decrements, and storage each take $O(|C| + |J| + P)$O(|C| + |J| + P). □□

An unchecked justification ($B(j) = \varnothing$B(j) = ∅) contributes to grounded reachability $D(M)$D(M) without entering J_EJE, whereas a checked zero-premise rule ($P_j = \varnothing, B(j) \neq \varnothing$Pj = ∅, B(j) ≠ ∅) fires unconditionally in $W(M, E)$W(M, E) once its bound obligation passes. Active nogoods on any derived configuration $X \subseteq C$X ⊆ C are evaluated after closure:

$$\mathrm{ActiveNogoods}(X) = \{n \in N \mid K_n \subseteq X\}.$$
$$\mathsf{ActiveNogoods(X) = \{n \in N \mid Kn \subseteq X\}.}$$

A nonempty $\mathrm{ActiveNogoods}(W(M, E))$ActiveNogoods(W(M, E)) reports the violated constraints and blocks publication of the candidate while preserving the underlying roots and alternative derivations.

3.3 Context semantics, labels, and exponential cost

Section titled "3.3 Context semantics, labels, and exponential cost"

Fix eligible positive rules J_*J∗, mandatory roots BB, optional assumption roots AA disjoint from BB, and applicable nogoods NN. For each assumption environment $E \subseteq A$E ⊆ A, define its positive closure, consistency predicate, and minimal label family by

$$C_E = \mathrm{closure}_{J_*}(B \cup E), \qquad \mathrm{Cons}(E) \iff \mathrm{ActiveNogoods}(C_E) = \varnothing,$$
$$\mathsf{CE = closure}_{J*}\mathsf{(B \cup E),} \qquad \mathsf{Cons(E) \iff ActiveNogoods(CE) = \varnothing,}$$

$$\mathrm{Labels}(c) = \min_{\subseteq}\left\{E \subseteq A \;\middle|\; \mathrm{Cons}(E) \wedge c \in C_E\right\}.$$
$$\mathsf{Labels(c) = \min_{\subseteq}\{E \subseteq A \mid Cons(E) \wedge c \in CE\}.}$$

Proposition 2. Nogood checks restricted to individual derivation trees do not establish root-context consistency.

Proof. Let

$$B = \{r\}, \qquad J_* = \{r \to a,\ r \to b\}, \qquad K = \{a, b\}.$$
$$\mathsf{B = \{r\},} \qquad \mathsf{J* = \{r \to a,\ r \to b\},} \qquad \mathsf{K = \{a, b\}.}$$

The derivation tree for aa visits $\{r, a\}${r, a} and the derivation tree for bb visits $\{r, b\}${r, b}; neither contains the nogood $K = \{a, b\}$K = {a, b}. Yet $C_\varnothing = \{r, a, b\} \supseteq K$C∅ = {r, a, b} ⊇ K, so every environment $E \subseteq A$E ⊆ A has $K \subseteq C_E$K ⊆ CE and no consistent environment exists. □□

Theorem 2. (i) Consistency is downward-closed in optional environments: $E' \subseteq E \wedge \mathrm{Cons}(E) \implies \mathrm{Cons}(E')$ E′ ⊆ E ∧ Cons(E) ⟹ Cons(E′). (ii) Every label in $\mathrm{Labels}(c)$Labels(c) is a consistent environment supporting cc, and every consistent environment EE supporting cc contains some $L \in \mathrm{Labels}(c)$L ∈ Labels(c); consequently cc admits consistent support iff $\mathrm{Labels}(c) \neq \varnothing$ Labels(c) ≠ ∅. (iii) For a finite nonempty target set TT, joint consistent support exists iff there is a selection $(L_t \in \mathrm{Labels}(t))_{t \in T}$(Lt ∈ Labels(t))t∈T satisfying

$$\mathrm{Cons}\Big(\bigcup_{t \in T} L_t\Big),$$
$$\mathrm{Cons}(t \in T \cup \mathrm{Lt}),$$

and for $T = \varnothing$T = ∅ joint consistent support exists iff $\mathrm{Cons}(\varnothing)$Cons(∅).

Proof. (i) For $E' \subseteq E$E′ ⊆ E, monotonicity of positive closure gives $C_{E'} \subseteq C_E$CE′ ⊆ CE; any nogood $K_n \subseteq C_{E'}$Kn ⊆ CE′ would also lie in C_E CE, so $\mathrm{Cons}(E) \implies \mathrm{Cons}(E')$Cons(E) ⟹ Cons(E′).

(ii) Soundness follows directly from the definition of $\mathrm{Labels}(c)$Labels(c). Given a consistent environment EE with $c \in C_E$c ∈ CE, the finite family $\{E' \subseteq E \mid \mathrm{Cons}(E') \wedge c \in C_{E'}\}${E′ ⊆ E | Cons(E′) ∧ c ∈ CE′} is nonempty and therefore has an inclusion-minimal member LL. Because any $E'' \subset L$E′′ ⊂ L lies inside EE, LL is also minimal in 2^A2A, so $L \in \mathrm{Labels}(c)$L ∈ Labels(c) and $L \subseteq E$L ⊆ E. Thus for any consistent EE, $c \in C_E \iff \exists L \in \mathrm{Labels}(c),\ L \subseteq E$c ∈ CE ⟺ ∃L ∈ Labels(c), L ⊆ E.

(iii) If a consistent EE supports every $t \in T$t ∈ T, part (ii) yields $L_t \subseteq E$Lt ⊆ E for each tt, and their union $U = \bigcup_{t \in T} L_t \subseteq E$U = ∪t∈T Lt ⊆ E is consistent by (i). Conversely, if $U = \bigcup_{t \in T} L_t$U = ∪t∈T Lt is consistent, monotonicity gives $t \in C_{L_t} \subseteq C_U$t ∈ CLt ⊆ CU for every $t \in T$t ∈ T. For $T = \varnothing$T = ∅, (i) implies that any consistent EE witnesses $\mathrm{Cons}(\varnothing)$Cons(∅). □□

Proposition 3. An $O(n)$O(n)-size positive justification system can have 2^n2n minimal labels for a single claim.

Proof. Let $A = \{a_i, b_i \mid 1 \le i \le n\}$A = {ai, bi | 1 ⩽ i ⩽ n}, $B = \varnothing$B = ∅, $N = \varnothing$N = ∅, with rules $a_i \to x_i$ai → xi, $b_i \to x_i$bi → xi ($1 \le i \le n$ 1 ⩽ i ⩽ n) and $\{x_1, \ldots, x_n\} \to z${x1, … , xn} → z. An environment $E \subseteq A$E ⊆ A derives zz iff $E \cap \{a_i, b_i\} \neq \varnothing$E ∩ {ai, bi} ≠ ∅ for every ii, and EE is inclusion-minimal iff $|E \cap \{a_i, b_i\}| = 1$|E ∩ {ai, bi}| = 1 for every ii. The monotone lineage formula and minimal-label count are therefore

$$\Phi_{\mathrm{mon}}(a, b) = \bigwedge_{i=1}^{n} (a_i \vee b_i), \qquad \big|\mathrm{Labels}(z)\big| = 2^n,$$
$$\Phi\mathrm{mon}(a, b) = \mathrm{i}{=}1 \wedge \mathrm{n}(\mathrm{ai} \vee \mathrm{bi}), \qquad \big|\mathrm{Labels}(z)\big| = 2\mathrm{n},$$

requiring $\Omega(n2^n)$Ω(n2n) symbols to materialize explicitly even though the factored circuit has size $O(n)$O(n). □□

For graphs with positive support cycles, JIRL retains the finite positive hypergraph or its least-fixed-point Boolean circuit rather than semiring provenance over $N[X]$N[X]: a self-loop $p \to p$p → p on root pp stabilizes at $\{p\}${p} under Boolean idempotence, whereas the formal power-series equation $p = a + p$p = a + p diverges over $N[X]$N[X]. Because explicit label materialization can grow exponentially, context enumeration is executed on demand under an explicit work and output budget, returning an `incomplete` report with discovered witnesses if the budget is exhausted.

3.4 Assurance and publication readiness

Section titled “3.4 Assurance and publication readiness”

Warrant $W(M, E)$W(M, E) is evaluated relative to the admitted roots RR, the assurance profile, and the current exact-request evidence in EE. Only `passed` reports admit a bound obligation into R_ERE or J_EJE; outcomes `refuted`, `incomplete`, `unsupported`, `error`, and `not_run`

do not discharge the obligation, while their distinct tags preserve diagnostic and recovery semantics (for example, distinguishing a budget timeout that invites resumption from an unsupported checker adapter). Because warrant propagates along any eligible derivation in J_EJE, a claim supported by a passing derivation remains warranted even when an alternative derivation route fails.

The publication readiness reducer checks five conditions: (1) structural and reference–closure validity of MM, (2) $R_C \subseteq W(M,E)$RC ⊆ W(M, E), (3) every required obligation in R_ORO has an admitted `passed` report, (4) $\mathrm{ActiveNogoods}(W(M,E)) = \varnothing$ ActiveNogoods(W(M, E)) = ∅, and (5) all decision consistency, resolution, and profile-specific independence requirements hold. Grounded reachability $D(M)$D(M), warranted closure $W(M,E)$W(M, E), active nogoods, check reports, and authorization decisions are exposed as separate inspection projections on the snapshot.

4. Compositional verification boundaries and executable realization

Section titled “4. Compositional verification boundaries and executable realization”

4.1 Selecting a composition rule

Section titled “4.1 Selecting a composition rule”

Proposition 4. Intent connected components and raw dependency–SCC cardinality are neither sufficient nor necessary criteria for choosing between joint state–space exploration and compositional contract verification.

Proof. Connect two independent Boolean state machines M_1, M_2M1, M2 to a shared parent rationale claim. Equip each M_iMi with a local safety invariant Φ_iΦi preserved by its own transitions and a frame condition stating that M_{3-i}M3−i does not modify M_iMi‘s state. Then $\Phi_1 \wedge \Phi_2$Φ1 ∧ Φ2 is discharged compositionally by local invariance plus framing even though the intent graph is connected. Conversely, let M_1, M_2M1, M2 be disconnected in the intent graph while both acquire a shared capacity-one resource omitted from the rationale edges; each local transition system satisfies its local specification, whereas the composed product violates mutual exclusion.

Similarly, a single component with transition sequence $\mathrm{idle} \to \mathrm{sent} \to \mathrm{acked}$idle → sent → acked forms a singleton SCC yet requires temporal behavioral verification. Conversely, an acyclic two-stage pipeline $\mathrm{producer} \to \mathrm{consumer}$ producer → consumer where the producer emits sendsend before closeclose while the consumer accepts closeclose before sendsend is already incompatible at its initial product state despite having no multi-node SCC. Finally, two components with state variables x, yx, y initialized to $0$0 that mutually observe each other while retaining $0$0 form a two-node interface SCC whose joint invariant $x = 0 \wedge y = 0$x = 0 ∧ y = 0 holds by local invariance and framing without product exploration. Behavioral contracts include transition-system models as well as pre/postcondition predicates. □□

For an obligation oo, define the semantic interference relation $\mathrm{interferes}_o(x, y)$interfereso(x, y) to hold when an admitted variation in xx can alter a state, event, resource, configuration, or query collection consumed in establishing yy‘s contribution to oo within the declared model horizon. Strongly connected components of $\mathrm{interferes}_o$interfereso help schedule verification tasks, while the obligation itself specifies the observable boundary, component and environment semantics, composition rule, and code-correspondence duty. Introducing a bounded queue between components introduces a stateful connector with its own ordering, capacity, and progress obligations.

4.2 Joint product exploration

Section titled “4.2 Joint product exploration”

Let each component ii be a labeled transition system $M_i = (S_i, I_i, \Sigma_i, \to_i)$Mi = (Si, Ii, Σi, →i) with initial states $I_i \subseteq S_i$Ii ⊆ Si, and let $P = (S_P, I_P, \Sigma_P, \to_P)$P = (SP, IP, ΣP, →P) be their declared parallel composition under explicit synchronization, interleaving, shared-state, and environment rules. Discharging a state invariant $\Phi \subseteq S_P$Φ ⊆ SP on PP requires

$$I_P \subseteq \Phi, \qquad \forall s \in \mathrm{Reach}(P),\ s \xrightarrow{a}_P s' \Longrightarrow (s \in \Phi \Rightarrow s' \in \Phi).$$

IP ⊆ Φ, ∀s ∈ Reach(P), s a

By induction on trace length, these two conditions establish $\mathrm{Reach}(P) \subseteq \Phi$Reach(P) ⊆ Φ, whereas any reachable state $s \notin \Phi$s ∉/ Φ yields a finite counterexample trace. Deadlock freedom ($\forall s \in \mathrm{Reach}(P) \setminus \mathrm{Term},\ \exists a, s',\ s \xrightarrow{a}_P s'$∀s ∈ Reach(P) ∖ Term, ∃a, s′, s a

---

P s′) and temporal liveness under stated fairness assumptions are specified as separate obligations. Over an explicit finite product graph, invariant exploration takes $O(|S_P| + |T_P|)$O(|SP| + |TP|) time and space after constructing PP (where $|S_P|$|SP| may grow as $\prod_i |S_i|$Π|Si|); when exploration is truncated by a depth or state bound without an analytical completeness threshold, the checker reports `incomplete`.

4.3 Sequential contracts, interface automata, and budget splitting

Section titled “4.3 Sequential contracts, interface automata, and budget splitting”

In a feed–forward functional pipeline, stage Hoare triples $\{P_i\}\ C_i\ \{Q_i\}$Pi Ci Qi compose sequentially once each implication $Q_i \implies P_{i+1}$ Qi ⟹ Pi+1 and frame condition are discharged; adding stage termination establishes total correctness for the pipeline.

For reactive protocols, an interface automaton $P = (S, s_0, I, O, H, T)$P = (S, s0, I, O, H, T) partitions actions into inputs II, outputs OO, and hidden internal steps HH, without requiring input–enabledness at every state [7]. Two interface automata P, QP, Q are composable when their hidden actions are disjoint from all actions of the partner and their input and output sets are pairwise disjoint; shared actions $\mathrm{shared}(P,Q) = (I_P \cap O_Q) \cup (O_P \cap I_Q)$shared(P, Q) = (IP ∩ OQ) ∪ (OP ∩ IQ) synchronize and become hidden in the product $P \otimes Q$P ⊗ Q. A product state $(s,t) \in \mathrm{Illegal}(P,Q)$(s, t) ∈ Illegal(P, Q) arises when one automaton enables a shared output aa that the partner does not enable as an input. Letting $\mathrm{OHpre}(U)$OHpre(U) denote product states that can reach UU via a single output or hidden step, define

$$\mathrm{Bad} = \mu X.\big(\mathrm{Illegal} \cup \mathrm{OHpre}(X)\big), \qquad \mathrm{Cmp} = S_{P\times Q} \setminus \mathrm{Bad}.$$
Bad = μX.(Illegal ∪ OHpre(X)), Cmp = SP×Q ∖ Bad.

Composable interface automata PP and QQ are optimistically compatible iff $(s_{0,P}, s_{0,Q}) \in \mathrm{Cmp}$(s0,P, s0,Q) ∈ Cmp [7, §§3.1–3.3]. Any state in the kk–th approximant of BadBad can be forced into $\mathrm{Illegal}$Illegal in kk autonomous output/hidden steps that no environment can block. Conversely, outside BadBad, an environment that accepts all component outputs and withholds all component inputs keeps execution within CmpCmp indefinitely (including at safe terminal states with no enabled inputs). Because optimistic compatibility asserts the *existence* of a helpful environment, deployment transfer requires discharging that the actual deployment environment satisfies the composite interface assumptions, and progress requires a separate non–silent environment obligation.

Interface refinement uses alternating simulation over ϵϵ–closures [7, §4]. With $\epsilon_P(s)$ϵP(s) denoting states reachable from ss by hidden steps, define

$$\begin{aligned}
\mathrm{In}_P(s) &= \bigcap_{u \in \epsilon_P(s)} \mathrm{enabledInput}_P(u),\\
\mathrm{Out}_P(s) &= \bigcup_{u \in \epsilon_P(s)} \mathrm{enabledOutput}_P(u),\\
\mathrm{Dest}_P(s,a) &= \{u' : \exists u \in \epsilon_P(s),\ u \xrightarrow{a}_P u'\}.
\end{aligned}$$
InP(s) OutP(s) DestP(s, a) = u∈ϵP(s) ∩ enabledInputP(u), = u∈ϵP(s) ∪ enabledOutputP(u), = {u′ : ∃u ∈ ϵP(s), u a

An alternating simulation $R \subseteq S_{\mathrm{abs}} \times S_{\mathrm{impl}}$R ⊆ Sabs × Simpl relates initial states, requires $I_{\mathrm{abs}} \subseteq I_{\mathrm{impl}}$Iabs ⊆ Iimpl and $O_{\mathrm{impl}} \subseteq O_{\mathrm{abs}}$Oimpl ⊆ Oabs, and satisfies for every $(s,t) \in R$(s, t) ∈ R:

$$\mathrm{In}_{\mathrm{abs}}(s) \subseteq \mathrm{In}_{\mathrm{impl}}(t), \quad \mathrm{Out}_{\mathrm{impl}}(t) \subseteq \mathrm{Out}_{\mathrm{abs}}(s),$$
Inabs(s) ⊆ Inimpl(t), Outimpl(t) ⊆ Outabs(s),

together with $\forall a \in \mathrm{In}_{\mathrm{abs}}(s) \cup \mathrm{Out}_{\mathrm{impl}}(t)$∀a ∈ Inabs(s) ∪ Outimpl(t) and $\forall t' \in \mathrm{Dest}_{\mathrm{impl}}(t,a)$∀t′ ∈ Destimpl(t, a), $\exists s' \in \mathrm{Dest}_{\mathrm{abs}}(s,a)$ ∃s′ ∈ Destabs(s, a) with $(s',t') \in R$(s′, t′) ∈ R. Substitutability in an environment context EE follows when EE is composable with the implementation and exercises only inputs present in I_{abs}Iabs.

For a quantitative end–to–end latency budget across stage timestamps $t_0 \le t_1 \le \cdots \le t_k$t0 ⩽ t1 ⩽ ⋯ ⩽ tk of the **same request**, stage bounds $t_i - t_{i-1} \le b_i$ti − ti−1 ⩽ bi that partition the full request path (including queueing, retries, synchronization, and scheduler wait at each handoff) telescope directly:

$$t_0 \le t_1 \le \cdots \le t_k, \qquad t_i - t_{i-1} \le b_i \quad (1 \le i \le k),$$
$$\sum_{i=1}^{k} b_i \le B \qquad \Longrightarrow t_k - t_0 \le B.$$

t0 ⩽ t1 ⩽ ⋯ ⩽ tk, ti − ti−1i=1Σk bi ⩽ B ⩽ bi (1 ⩽ i ⩽ k), ⟹ tk − t0 ⩽ B.

4.4 Circular assume–guarantee reasoning without vacuity

Section titled “4.4 Circular assume–guarantee reasoning without vacuity”

Naive circular implications $A_i \wedge L_i \Rightarrow G_i$Ai ∧ Li ⇒ Gi and $\bigwedge_j G_j \Rightarrow A_i$∧j Gj ⇒ Ai admit a vacuous assignment in which every A_iAi and G_iGi is identically false. Breaking this circularity requires well–founded induction along execution prefixes.

For any infinite execution σσ and prefix–closed safety property PP (where the empty prefix satisfies $P[0]$P[0]), let $P[k]$P[k] denote that the length–kk prefix of σσ satisfies PP. Given environment constraint EE, concrete component implementations L_iLi, safety guarantees G_iGi, and safety assumptions A_iAi, the finite–prefix circular safety obligations are:

1. **Local one–step extension:** on every execution of L_iLi under EE and the declared open–component semantics,

$$A_i[k] \Longrightarrow G_i[k+1] \qquad (\forall k \ge 0).$$
Ai[k] ⟹ Gi[k + 1] (∀k ⩾ 0).

2. **Assumption compatibility:** on every admitted prefix under EE,

$$\bigwedge_j G_j[k] \Longrightarrow A_i[k] \qquad (\forall i,\ \forall k \ge 0).$$
j∧ Gj[k] ⟹ Ai[k] (∀i, ∀k ⩾ 0).

3. **Semantic composition:** every execution of the composed implementation under EE projects to executions of each L_iLi satisfying the framing, shared–variable, and hiding side conditions.

Lemma 3 (Non–vacuous circular safety). Under duties (1)—(3), every execution of the composed implementation under EE satisfies the safety property $\bigwedge_i G_i$∧i Gi.

Proof. Fix a composed execution σσ under EE. By prefix closure of safety properties, $G_i[0]$Gi[0] holds for all ii. Assuming $\bigwedge_j G_j[k]$∧j Gj [k], duty (2) yields $A_i[k]$Ai[k] for all ii, and duty (1) applied via duty (3) yields $G_i[k+1]$Gi[k + 1] for all ii. Induction on kk establishes $G_i[k]$ Gi[k] for every prefix length $k \ge 0$k ⩾ 0, which by definition of a safety property implies that σσ satisfies every G_iGi. Because $A_i[0]$Ai[0] holds at the base step, duty (1) immediately forces $G_i[1]$Gi[1] on the initial transition, preventing vacuous circularity. □□

When a specification requires general TLA properties combining safety and liveness, an obligation adapter can invoke the full Abadi—Lamport decomposition theorem [8, §4.3]. With $\mathrm{cl}(F)$cl(F) denoting the safety closure of FF and $F^{+\upsilon}$F+v denoting the property that permits a first violation of FF and thereafter freezes state tuple υv, the rule discharges

$$\models \bigwedge_j \mathrm{cl}(G_j) \quad \Rightarrow A_i,$$
$$\models \mathrm{cl}(A_i)^{+v} \wedge \mathrm{cl}(L_i) \quad \Rightarrow \mathrm{cl}(G_i),$$
$$\models A_i \wedge L_i \quad \Rightarrow G_i,$$

⊨ j ∧ cl(Gj) ⊨ cl(Ai)+v ∧ cl(Li) ⊨ Ai ∧ Li ⇒ Ai, ⇒ cl(Gi), ⇒ Gi,

subject to the environment–closure, variable–hiding, and refinement–mapping side conditions of [8].

4.5 Vertical refinement and receptiveness

Section titled “4.5 Vertical refinement and receptiveness”

Let $M_A = (S_A, I_A, \Sigma_A, \rightarrow_A)$MA = (SA, IA, ΣA, →A) and $M_C = (S_C, I_C, \Sigma_C, \rightarrow_C)$MC = (SC, IC, ΣC, →C) be abstract and concrete transition systems connected by a state abstraction map $\alpha: S_C \to S_A$α : SC → SA and action map $\lambda: \Sigma_C \to \Sigma_A \cup \{\epsilon\}$λ : ΣC → ΣA ∪ {ϵ}. On the reachable concrete state space, require $\alpha(I_C) \subseteq I_A$α(IC) ⊆ IA and the step simulation condition

$$s \xrightarrow{a}_C s' \Longrightarrow \begin{cases} \alpha(s) \xrightarrow{\lambda(a)}_A \alpha(s'), & \lambda(a) \neq \epsilon, \\ \alpha(s) = \alpha(s'), & \lambda(a) = \epsilon. \end{cases} \qquad \text{(R1)}$$

s a

Lemma 4 (Finite–trace safety preservation). Initial mapping $\alpha(I_C) \subseteq I_A$α(IC) ⊆ IA and (R1) map every concrete finite execution, after erasing ϵϵ–steps, to a valid abstract execution with matching abstract endpoints, preserving all universal finite–trace safety properties from M_AMA to M_CMC.

Proof. A zero–step concrete execution maps to an abstract initial state in I_AIA. Inductively, if a concrete prefix ending at ss maps to an abstract execution ending at $\alpha(s)$α(s), a concrete step $s \xrightarrow{a}_C s'$s a

---

C s′ either extends the abstract execution by $\alpha(s) \xrightarrow{\lambda(a)}_A \alpha(s')$α(s) λ(a)

A α(s′) when $\lambda(a) \neq \epsilon$λ(a) = ϵ, or leaves the abstract endpoint $\alpha(s) = \alpha(s')$α(s) = α(s′) unchanged when $\lambda(a) = \epsilon$λ(a) = ϵ. □□

Because (R1) alone is satisfied by a trivial concrete system with no enabled transitions, interface substitutability adds an **input receptiveness** obligation: for every reachable concrete state ss and every abstract input aa enabled at $\alpha(s)$α(s), the concrete input adapter must enable at least one corresponding concrete input transition from ss whose responses satisfy (R1). Likewise, preserving response liveness across potentially silent concrete steps ($\lambda(a) = \epsilon$λ(a) = ϵ) requires a well–founded rank on consecutive silent steps together with the applicable fairness assumptions.

4.6 Leaf realization

Section titled “4.6 Leaf realization”

At the bottom of a refinement hierarchy, a leaf realization obligation relates pinned program bytes PP to a leaf specification MM under an explicit compilation/runtime environment and observation adapter $\mathrm{implements}(P, M)$implements(P, M):

- **Checked generation or translation validation.** A verified code generator or per–artifact translation validator establishes $\mathrm{implements}(P, M)$implements(P, M) for a restricted modeling fragment under the pinned toolchain contract.
- **Independently authored code with scoped conformance.** An execution trace adapter maps concrete runtime events to actions of M M. Property–based and replay tests establish conformance over the exercised finite trace set; exhaustive bounded model checking

establishes conformance up to the declared state and input bounds; and deductive program verification establishes universal trace inclusion $\mathrm{Traces}(P) \subseteq \mathrm{Traces}(M)$Traces(P) ⊆ Traces(M) relative to the formalized language semantics.

5. Incremental evolution and transactional publication

Section titled “5. Incremental evolution and transactional publication”

5.1 Applicability, early cutoff, and from-scratch consistency

Section titled “5.1 Applicability, early cutoff, and from-scratch consistency”

Proposition 5. Neither unchanged artifact identity nor unchanged authored interface digest suffices on its own to certify check reuse.

Proof. Let $f() = 0$f() = 0. Against obligation $o_1 : \mathrm{result} \geq 0$o1 : result ⩾ 0, ff passes; when the obligation is strengthened to $o_2 : \mathrm{result} > 0$ o2 : result > 0 without modifying ff‘s bytes, keying a cache solely by artifact digest falsely reuses `passed`. Second, let $f() = 1$f() = 1 and $g() = -1$g() = −1 share the authored interface signature $() \to \mathrm{int}$() → int with postcondition $\mathrm{result} \geq 0$result ⩾ 0; their interface digests are identical even though gg violates the interface contract, so interface-based cutoff requires verified provider conformance. Third, even when $f_1() = 1$f1() = 1 and $f_2() = 2$f2() = 2 both conform to $\mathrm{result} \geq 0$result ⩾ 0, a consumer check testing $\mathrm{result} = 1$result = 1 distinguishes them because its observation is not factored through the coarse interface predicate. □□

Define incremental evaluation over a finite DAG VV of **verification units** (where cyclic component dependencies are encapsulated within a single joint unit or evaluated via a fixed-point operator of proved determinism). With external inputs xx pinned, a deterministic cold evaluator computes in topological order

$$r_v = f_v\big(p_v(x, (r_u)_{u \in \mathrm{pred}(v)})\big),$$
rv = fv(pv(x, (ru)u∈pred(v))),

where $p_v$pv is a complete observational projection of the inputs and predecessor results consumed by vv (including any queried collection memberships), and $f_v$fv encapsulates checker identity, obligation specification, configuration, budget, assurance policy, and the semantic verdict projection.

Theorem 3. Suppose (a) the verification-unit DAG VV, evaluator functions $f_v$fv, and complete observational projections $p_v$pv are validated for the candidate input; (b) cold evaluation is deterministic and terminating; (c) each cache entry for vv stores a prior evaluation of the identical $f_v$fv at a recorded projection value, or a verified reuse certificate establishes $f_v(p_v(\mathrm{new})) = r_v^{\mathrm{cached}}$fv(pv(new)) = rvcached; (d) the incremental evaluator traverses affected units in topological order and reuses r_v^{cached}rvcached only when $p_v(\mathrm{new}) = p_v(\mathrm{cached})$pv(new) = pv(cached) or a verified reuse certificate holds; and (e) the readiness reducer is a deterministic function of exact material, unit verdicts, assurance profile, and environment. Then incremental evaluation produces the exact cold verdict vector $(r_v)_{v \in V}$(rv)v∈V and readiness outcome. In particular, change propagation cuts off at a semantic interface whenever the new provider is proved to conform to that interface and every downstream consumer’s complete projection $p_v$pv factors exclusively through that interface.

Proof. We induct along a topological order of VV. For any source unit vv, $p_v$pv depends only on pinned external inputs xx, which coincide in cold and incremental runs. For an internal unit vv, the induction hypothesis gives equality of predecessor results $(r_u)_{u \in \mathrm{pred}(v)}$(ru)u∈pred(v), so the newly computed projection $p_v(\mathrm{new})$pv(new) equals the cold projection. On a cache miss, incremental evaluation invokes $f_v(p_v(\mathrm{new}))$fv(pv(new)), matching the cold result; on an equal-projection hit or certified reuse, condition (c) guarantees equality with $f_v(p_v(\mathrm{new}))$fv(pv(new)). Feeding the identical result vector into the deterministic reducer (e) yields the identical closure, conflict set, and readiness verdict. When a provider change preserves a verified semantic interface through which a consumer’s projection $p_v$pv factors completely, $p_v(\mathrm{new}) = p_v(\mathrm{cached})$pv(new) = pv(cached) holds and propagation terminates at the interface boundary. □□

Each hypothesis in Theorem 3 is load-bearing: if a checker inspects unprojected implementation details, if a stochastic checker is run without a pinned random seed, or if wall-clock timeouts vary outside the modeled projection $p_v$pv, cold and incremental runs can diverge

on the unmodeled channel. When an earlier `EvidenceRecord` is reused via a projection or certificate check, a fresh `EvidenceRecord` binds the new exact `CheckRequest` to the earlier `basis` reference.

5.2 Deterministic orchestration and phase-structured outcomes

Section titled “5.2 Deterministic orchestration and phase-structured outcomes”

Given a candidate interpretation, choice policy ππ, authority policy αα, and recorded external checker/provider observations ωω, the orchestration reducer is deterministic. The evolution interface partitions outcomes by protocol phase:

$$\text{evolve}(G, \Delta I, \pi, \alpha; \omega) \in \Big\{ \begin{array}{l} \text{Published}(S),\ \text{Halted}(\text{phase}, \text{cause}, \text{evidence}), \\ \text{PublicationIndeterminate}(\text{attempt}, \text{observations}) \Big\}, \end{array}$$

evolve(G, ΔI, π, α; ω) ∈ {Published(S), Halted(phase, cause, evidence), PublicationIndeterminate(attempt, obser

where $\text{phase} \in \{\text{Construct}, \text{Evaluate}, \text{Authorize}, \text{Publish}\}$phase ∈ {Construct, Evaluate, Authorize, Publish}. Distinguishing `PublicationIndeterminate` from `Halted` preserves the store boundary condition in which an atomic ref update may have succeeded before an acknowledgement or readback failure occurred.

```
evolve(bases, intent, policy, authority, observations):
  Construct: select interpretation and assemble a complete immutable Material
             (choices, obligation revisions, retirements, and merge reconciliation)
             or return Halted(Construct, invalid_input | unresolved, diagnostics)
  Evaluate:  validate schema and closure; bind exact CheckRequests; execute or certify reuse;
             freeze E; compute D(M), W(M, E), ActiveNogoods(W), and readiness
             or return Halted(Evaluate, blocked, E)
  Authorize: diff protected accepted items and roles against all bases;
             request coverage for (bases, M, E, dispositions) from authority provider
             or return Halted(Authorize, unauthorized | provider_indeterminate, receipt)
  Publish:   persist immutable objects and seal Snapshot S;
             execute atomic expected-head compare-and-swap with durable transition log
             return Published(S) | Halted(Publish, stale_base, head)
                  | PublicationIndeterminate(S, observations)
```

The candidate MM is sealed upon entering `Evaluate`: any modification to claims, selections, or obligations constructs a new `Material` and restarts at `Construct`. Similarly, reconciling a `stale_base` conflict constructs and evaluates a new merged candidate before requesting fresh authorization against the updated parent heads.

5.3 Read skew, semantic merge, and protected changes

Section titled “5.3 Read skew, semantic merge, and protected changes”

Proposition 6. Resolving dependencies by independently selecting the newest revision of each lineage can violate both cross-claim invariants and structural acyclicity invariants.

Proof. Consider state variables (x, y)(x, y) subject to invariant $x = y$x = y from base $(0, 0)$(0, 0). Branch PP publishes $(1, 1)$(1, 1) and branch QQ publishes $(2, 2)$(2, 2), with timestamps ordered so $P.x$P.x is newer than $Q.x$Q.x while $Q.y$Q.y is newer than $P.y$P.y. Selecting the newest revision of each lineage yields $(1, 2)$(1, 2), violating $x = y$x = y. Similarly, for an application imposing acyclicity on a relation over vertices $\{p, q\}${p, q}, let one branch add $p \to q$p → q and another add $q \to p$q → p; each branch is acyclic, whereas taking the union of newest lineage revisions creates the cycle $p \leftrightarrows q$p ⇆ q. Pinned content references in SS prevent both mixtures. □□

Lemma 5 (Publication safety). Under immutable retained objects, validated reference closure, a sound readiness reducer, an authority provider covering all modified or retired protected base items and roles, and a linearizable expected-head compare-and-swap that records durable transition receipts, every `Published(S)` transition installs an evaluated and authorized snapshot SS, concurrent pinned readers observe a single coherent snapshot, and every change to a protected base item or role is accompanied by an authorized `AcceptedDisposition`.

Proof. Pre–publication validation binds SS‘s exact `material`, `evaluation`, and `parents` references to those covered by AA, and readiness was established on those identical bytes. The atomic compare–and–swap on the named ref either aborts without mutation when the current head differs from parents[0]parents[0] or linearizes at a single transition installing SS. For every protected item and role in the base snapshots (`roots`, `requiredClaims`, `requiredObligations`, `justifications`, `decisions`, `nogoods`), the base–diff check verifies either exact retention in the same role or an explicit `AcceptedDisposition` covered by AA. Pinned readers resolve SS in $O(1)$O(1) store operations by Lemma 2. □□

Checking protected roles as well as object existence is essential: keeping an obligation in `obligations` while removing it from `requiredObligations`, or demoting a root in `roots` to an ordinary claim in `claims`, withdraws an accepted guarantee and therefore requires an explicit `AcceptedDisposition`. Likewise, expected–head compare–and–swap serializes updates to a single ref, while semantic compatibility across merged branches (for example, when two branches independently consume $10$10 units of a shared capacity–$10$10 budget $x + y \leq 10$x + y ⩽ 10 from base $(0, 0)$(0, 0)) is enforced by re–evaluating the joint obligation on the merged `Material`.

6. Feasibility, complexity, and falsification

Section titled “6. Feasibility, complexity, and falsification”

6.1 Algorithmic costs and their boundaries

Section titled “6.1 Algorithmic costs and their boundaries”

Let $c = |C|$c = |C|, $j = |J|$j = |J|, $p = \sum_j |P_j|$p = Σj |Pj|, $n = |N|$n = |N|, $q = \sum_n |K_n|$q = Σn |Kn|, $a = |A|$a = |A|, $o = |O|$o = |O|, and bb denote the encoded byte length of inspected objects. Solver and checker execution times enter as explicit additive terms.

Operation	Time	Working / storage space	Scope condition
Decode, canonicalize, and validate	$\Omega(b)$Ω(b) plus canonical sort cost	Encoded objects and index tables	Fixed schema and closed references
Grounded closure $D(M)$D(M)	$O(c + j + p)$O(c + j + p)	$O(c + j + p)$O(c + j + p)	Counter worklist on canonical premise sets
Warranted closure $W(M, E)$W(M, E)	$O(c + j + p + o)$O(c + j + p + o) plus check execution	Worklist and outcome index	Frozen eligibility EE
Active nogood check	$O(n + q)$O(n + q)	$O(n)$O(n) counters plus membership bitmap	Subset test against derived configuration
Single–environment consistency $\mathrm{Cons}(E)$Cons(E)	$O(c + j + p + n + q)$O(c + j + p + n + q)	$O(c + j + p + n)$O(c + j + p + n)	Fixed eligible rules and nogoods
Exhaustive environment oracle (all $E \subseteq A$E ⊆ A)	$O(2^a(c + j + p + n + q) + a2^a)$ O(2a(c + j + p + n + q) + a2a)	$O(2^a)$O(2a) table plus emitted labels	Subset DP for inclusion–minimality
Explicit minimal labels $\mathrm{Labels}(z)$ Labels(z)	$\Omega(n_0 2^{n_0})$Ω(n02n0) symbols (Prop. 3)	Output size if materialized	Worst–case output lower bound for n_0 n0 pairs
Budgeted why/context query	Bounded by declared work/output budget	Declared frontier and output buffer	Returns `incomplete` on budget exhaustion
Scoped absence scan	$O(\lvert I_\sigma \rvert \cdot T_q)$O(|Iσ| Tq) plus payload bytes	Match buffer and iterator state	Decidable predicate of per–claim cost T_qTq
Explicit product invariant check	$O(\lvert S_P \rvert + \lvert T_P \rvert)$O(|SP| + |TP|)	$O(\lvert S_P \rvert + \lvert T_P \rvert)$O(|SP| + |TP|)	$\lvert S_P \rvert$ |SP| can grow as $\prod_i \lvert S_i \rvert$ Π|Si|
Untimed IA optimistic compatibility	Product build $+O(\lvert S_{P\times Q} \rvert + \lvert T_{P\times Q} \rvert)$ +O(|SP×Q| + |TP×Q|)	Product graph and backward OHpre OHpre index	Finite safety game [7]
Compositional AG / refinement check	Sum of component obligation check costs	Checker–dependent	Requires user/adapter–supplied assumptions
Pinned snapshot root resolution	$O(1)$O(1) indexed store lookups	Root envelope bytes	Full closure traversal of kk objects is $O(k)$O(k)
Snapshot diff	Proportional to visited index nodes and delta	Traversal frontier plus emitted diff	Structural sharing bounds unchanged subtrees

Operation	Time	Working / storage space	Scope condition
Certified incremental evaluation	Dependency scan + projection check + affected $f_v$fv	Dependency graph and cache entries	Theorem 3 guarantees cold equivalence; worst-case impact is $O(\mid V \mid)$O(|V|)
Expected-head publication	Object write cost + atomic ref CAS	New immutable objects and transition log	Linearizable ref store with durable log

The $O(2^a(c + j + p + n + q) + a2^a)$O(2a(c + j + p + n + q) + a2a) exhaustive label oracle evaluates C_ECE and $\mathrm{Cons}(E)$Cons(E) for each $E \subseteq A$E ⊆ A and applies boolean subset dynamic programming to filter out environments with a supporting proper subset, providing a simple reference oracle for differential testing on small assumption sets. While the finite positive support kernel runs in linear time $O(c + j + p)$O(c + j + p), end-to-end evaluation time is governed by the attached verification obligations and by the breadth of obligations affected by a change.

6.2 Authoring and maintenance cost

Section titled “6.2 Authoring and maintenance cost”

The practical adoption bottleneck for design-rationale and formal-refinement systems is the ratio of authoring and maintenance effort to diagnostic and verification value. JIRL bounds capture scope through **obligation-scoped recording**: claims, grouped justifications, decision records, and interference links are recorded when they feed an active verification obligation, impact query, protected authority gate, or required architectural decision record.

Obligations support three complementary assurance practices that may be combined on the same subsystem:

- **Argumentative:** structured claims, grouped premises, and explicit trade-off decisions (`DecisionRecord`), with empirical subclaims bound to test or query evidence.
- **Contractual:** explicit interface specifications and environment assumptions (such as Hoare pre/postconditions or interface automata) governing component boundaries.
- **Mechanical:** automated proof, model-checking, or conformance test obligations discharging behavioral refinement and invariance properties.

With kk denoting the number of recorded obligation and decision events, $u_i$ui the authoring, review, and disposition cost of event ii, vv the checker integration cost, and ss the saved debugging and rework cost, the net engineering overhead is

$$\text{overhead} = \sum_{i=1}^{k} u_i + v - s.$$

overhead = i=1Σk ui + v − s.

Enforcing a recording budget $k \leq k_{\max}$k ⩽ kmax bounds allocated authoring effort at the cost of leaving unrecorded subsystems outside the ledger’s coverage scope.

6.3 Epistemic limits and deployment profiles

Section titled “6.3 Epistemic limits and deployment profiles”

Formal kernel guarantees operate within three foundational boundaries: (1) adequacy of formal specifications with respect to informal human intent remains a domain-validation problem [3,12]; (2) physical interference channels outside the declared model horizon (such as unmodeled shared hardware, timing channels, or operating-system state) are invisible to model-level checks; and (3) undecidable properties of general Turing-complete programs require conservative abstraction, interactive proof, or bounded testing.

Within those boundaries, deployments can combine two evaluation strategies with optional modeling restrictions:

Deployment profile	Mechanism and primary benefit	Primary cost or vulnerability
A: Whole-candidate cold evaluation	Re-evaluates all active requests on each candidate; minimal adapter complexity	Repeated execution of expensive checks

Deployment profile	Mechanism and primary benefit	Primary cost or vulnerability
B: Certified incremental evaluation	Reuses prior outcomes via complete projections $p_v$pv and interface certificates (Theorem 3)	Requires sound read-set tracking and provider conformance proofs
C: Restricted derivation DSL	Constrains component models so code generation or translation validation discharges leaf realization	Higher upfront modeling cost and restricted language expressiveness

6.4 Finite counterexamples and verification plan

Section titled “6.4 Finite counterexamples and verification plan”

Twenty-one machine-checked finite witness tests accompany this report in `jirl-cold-replay.jsonl` (9 records), `jirl-convergence-replay.jsonl` (4 records), and `jirl-paper-probes.jsonl` (8 records). Each witness constructs a concrete finite instance and verifies the corresponding proposition or counterexample:

- **Support and consistency witnesses (`P1`—`P4`, `P10`—`P13`, `JIRL-P14`—`JIRL-P16`):** nonmonotonicity and missing least fixed point of the locally guarded operator T_RTR (Proposition 1); order-independent linear-time convergence of F_MFM and $F_{M,E}$FM,E in the presence of multiple fixed points and cycles (Theorem 1); single-proof-tree nogood failure versus full-closure consistency (Proposition 2); downward-closed consistency and union consistency of minimal labels (Theorem 2); exponential 2^n2n minimal-label growth for $n \in \{1, \ldots, 7\}$n ∈ {1, … , 7} yielding $128$128 minimal labels at $n = 7$n = 7 (Proposition 3); checked zero-premise rules; and defeated scoped-absence roots.
- **Persistence, composition, and incremental reuse witnesses (`P5`—`P9`, `JIRL-P17`—`JIRL-P21`):** acyclic reference construction and topological stratification (Lemma 1); decision selection invariants (§2.3); read-skew invariant violations and cycle creation under per-lineage newest selection (Proposition 6); safe terminal states in interface automata compatibility (§4.3); failure of artifact-digest and unverified interface-digest caching, sensitivity of consumers to unprojected provider behaviors (Proposition 5); empty-query collection dependencies; and exact agreement between cold and projection-cached incremental evaluation across 16 test configurations (Theorem 3).

Prospective implementation verification extends these finite witnesses with a property-based differential test suite that generates random finite materials (including zero-premise rules, support cycles, alternative derivations, empty nogoods, and mutating query inventories) and checks: (1) counter-worklist closure against exhaustive least-closed-set enumeration, (2) computed minimal labels against 2^A2A subset dynamic programming, (3) incremental evaluation verdicts against cold evaluation on identical inputs, and (4) linearizable expected-head publication under simulated concurrent writers and lost acknowledgements.

6.5 Four-arm discriminating empirical protocol

Section titled “6.5 Four-arm discriminating empirical protocol”

Testing whether JIRL’s structural invariants yield net engineering value requires comparing four capability bundles over shared software-evolution histories:

Arm	Capability bundle
Arm 0: Capable baseline	Versioned source and contracts, build-graph dependency tracking, automated test suites, ADRs, and scoped codebase queries
Arm 1: Flat-parent ledger	Recorded claims and evidence linked by ungrouped parent edges under a fixed global conjunction/disjunction convention
Arm 2: Grouped positive support	Grouped AND/OR justifications (JJ) and obligations (OO) without scoped absence queries or nogoods (NN)
Arm 3: Full JIRL architecture	Grouped support, explicit decision records, stratified absence queries, nogoods, projection-based reuse, and atomic snapshot publication

To isolate representation effects from authoring variance, Arms 1—3 are evaluated wherever possible by projecting the same frozen, independently authored corpus of claims and obligations into each arm’s schema; when authoring effort differs across arms, capture and maintenance time are measured explicitly.

Corpus and witness scenarios. An initial ecological pilot tracks a single subsystem history comprising roughly 30—60 claims across 5—10 evolution steps, seeded with replayable witness scenarios for: absence invalidation upon claim insertion; premise withdrawal with surviving alternative derivations; hidden shared-resource contention; acyclic protocol-order mismatch; obligation strengthening over unchanged code bytes; semantic merge violating a shared capacity budget; rejected-but-feasible design options; bounded versus universal evidence scopes; and authorized retirement of protected obligations. Multi-history confirmatory studies size their sample count via predeclared power or precision targets.

Ground truth and metrics. Before running the arms, independent domain reviewers freeze the ground-truth classification of each obligation at each evolution step, defining `needs_attention` whenever an obligation's property is violated or its prior evidence ceases to apply under unchanged-input or certified-reuse rules. Recorded metrics per witness family are: invalidation precision and recall, explanation accuracy on a graded rubric of "why / what-if" queries, false guarantee assertions, detected cross-branch conflicts, checker execution cost, and human authoring/review time.

Five predeclared falsification criteria govern interpretation of the empirical results:

- **F1 — Overall architectural net benefit.** If Arm 0 is non-inferior to Arm 3 on invalidation recall and explanation accuracy within predeclared margins while incurring lower total authoring and operating cost, the ledger's added structure does not justify its overhead on that class of projects.
- **F2 — Scoped absence and nogoods.** If Arm 2 is non-inferior to Arm 3 on corpora containing absence-invalidation and cross-branch constraint witnesses at lower cost, explicit absence queries and nogoods are unnecessary for that workload.
- **F3 — Conjunctive/disjunctive grouping.** If Arm 1 is non-inferior to Arm 2 on premise-withdrawal and alternative-derivation witnesses, grouped hyperedges do not earn their representation cost over flat dependency links.
- **F4 — Soundness criterion.** Any instance in which Arm 3 certifies a required obligation as warranted and ready using stale, inapplicable, or incomplete evidence is a soundness failure of the implementation or model horizon, evaluated independently of average productivity metrics.
- **F5 — Interference-guided scheduling.** If the SCC decomposition of interferes_ointerfereso degenerates into all singletons or a single global component across representative changes, or fails to reduce verification scheduling cost relative to Arm 0, the interference-graph scheduling heuristic is rejected while retaining the underlying obligation calculus.

6.6 Conclusion

Section titled "6.6 Conclusion"

JIRL assigns each software-evolution question to a distinct mathematical boundary: positive closure $D(M)$D(M) and $W(M, E)$W(M,E) determine which derivations reach a claim; full-closure nogood evaluation ActiveNogoodsActiveNogoods determines whether premises can co-hold; exact check reports record what was verified and in what scope; action-specific authority records which protected obligations and roles may change; and content-addressed snapshots pin coherent configurations across time. At the same time, our complexity bounds and counterexamples establish the exact limits of the formalism: minimal assumption labels can grow as 2^n2n, graph topology cannot substitute for behavioral composition proofs, and incremental cutoff requires complete observational projections. A compact reference implementation of the whole-candidate evaluator and atomic snapshot store, paired with the differential oracle and the four-arm empirical protocol, provides a direct path to testing these guarantees in practice.

References

Section titled "References"

1. Johan de Kleer. *An Assumption-Based TMS*. Artificial Intelligence 28, 1986. Local: `references/library/pdf/dekleer-1986-atms.pdf`. David McAllester. *Truth Maintenance*. AAAI, 1990. Local: `references/library/pdf/mcallester-1990-truth-maintenance.pdf`.
2. Todd J. Green, Grigoris Karvounarakis, and Val Tannen. *Provenance Semirings*. PODS, 2007. Author-hosted manuscript. Local: `references/library/pdf/green-2007-provenance-semirings.pdf`.
3. Axel van Lamsweerde. *Goal-Oriented Requirements Engineering: A Guided Tour*. RE, 2001. Local: `references/library/pdf/van-lamsweerde-2001.pdf`.
4. Nam Pyo Suh. *Axiomatic Design*. MIT course material, 2004; theory developed in *The Principles of Design* (1990) and *Axiomatic Design: Advances and Applications* (2001). Local: `references/library/pdf/suh-2004-axiomatic-design.pdf`.

5. Tyson R. Browning. *Applying the Design Structure Matrix to System Decomposition and Integration Problems: A Review and New Directions*. IEEE Transactions on Engineering Management 48(3), 2001. Local: `references/library/pdf/browning-2001-dsm.pdf`.
6. Werner Kunz and Horst W. J. Rittel. *Issues as Elements of Information Systems*. Working Paper 131, University of California, Berkeley, 1970. Local: `references/library/pdf/kunz-rittel-1970-ibis.pdf`.
7. Luca de Alfaro and Thomas A. Henzinger. *Interface Automata*. ESEC/FSE, 2001, 109–120. DOI. Local: `references/library/pdf/dealfaro-henzinger-2001-interface-automata.pdf`.
8. Martín Abadi and Leslie Lamport. *Conjoining Specifications*. ACM TOPLAS 17(3), 1995, 507–534. Local: `references/library/pdf/abadi-lamport-1995-conjoining-specifications.pdf`.
9. Martín Abadi and Leslie Lamport. *The Existence of Refinement Mappings*. Theoretical Computer Science 82(2), 1991, 253–284. Local: `references/library/pdf/abadi-lamport-refinement.pdf`.
10. James R. Driscoll, Neil Sarnak, Daniel D. Sleator, and Robert E. Tarjan. *Making Data Structures Persistent*. Journal of Computer and System Sciences 38(1), 1989. Author-hosted manuscript. Local: `references/library/pdf/driscoll-1989-persistence.pdf`.
11. Matthew A. Hammer, Khoo Yit Phang, Michael Hicks, and Jeffrey S. Foster. *Adapton: Composable, Demand-Driven Incremental Computation*. PLDI, 2014. Author-hosted manuscript. Local: `references/library/pdf/hammer-2014-adapton.pdf`.
12. Pamela Zave and Michael Jackson. *Four Dark Corners of Requirements Engineering*. ACM TOSEM 6(1), 1997. Local: `references/library/pdf/zave-jackson-1997.pdf`.
13. Ira D. Baxter. *Design Maintenance Systems*. Communications of the ACM 35(4), 1992, 73–89. Author-hosted manuscript. Local: `references/library/pdf/baxter-1992-design-maintenance.pdf`.